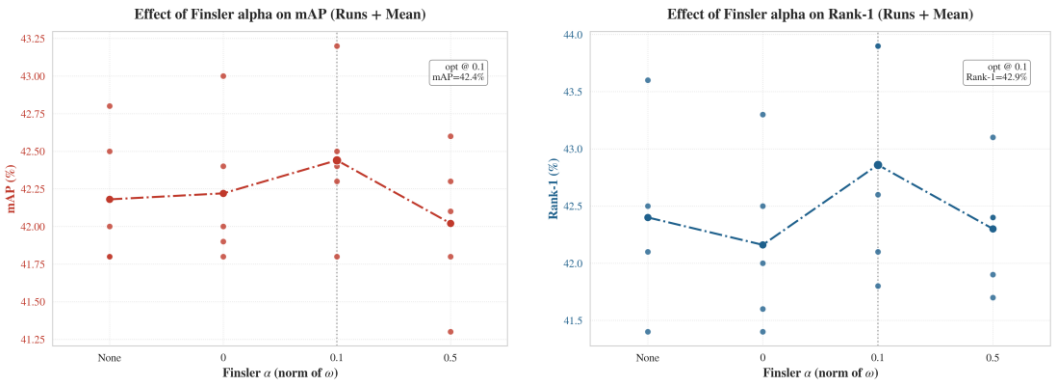
Finsler Alpha Sweep with Mean and Variance



- [mAP] Baseline and alpha=0 matches in terms of the mean value and variance
- [Rank-1] Still slight difference in mean and variance between baseline and sanity check

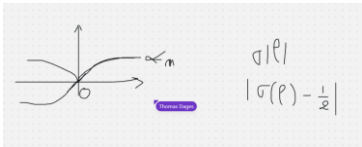
Both graphs point towards the initial conjecture that an alpha value of 0.1 appears optimal for this dataset configuration

- In the Euclidean case, there is a high variance
- In alpha=0.1, seem to concentrate around higher values
- Benefit is not that clear, difficult to see any contributions

- Not clear why we would hope for something better

Learnable Alpha – Model Introduction

Alpha Parameter



$0 < \alpha_{max} < 0$ be the maximum scaling factor (default 1.0).

$\rho \in R$ be the **learnable** logit parameter.

$$\alpha = \boxed{\alpha_{max}} \cdot \boxed{\sigma}\left(\frac{\boxed{\rho}}{\boxed{T}}\right)$$

$\sigma(z) = \frac{1}{1 + e^{-z}}$ be the sigmoid function.

$T > 0$ be the temperature scaling factor (default 1.0).

Why introduce the temperature hyperparameter?

- Control gradient sensitivity: Gradient of alpha w.r.t. raw pho is proportional to 1/T.
    - Since it is updated by the same global optimizer, the global step size might be too large/small for this single scalar.
    - Adjusting T acts as a local learning rate modifier
        - High T (> 1): Dampens the gradients
            - allows alpha to change slowly and smoothly, preventing it from oscillating wildly / saturating immediately
            - Smooths loss landscape w.r.t alpha, prevents optimization from being stuck in sharp local minima early in training
        - Low T (< 1): Amplified gradients -> forces alpha to converge quickly to a specific value
- Mitigating Vanishing Gradients (Sigmoid Saturations)
    - Sigmoid becomes very flat when absolute value of input (logits rho) is large
    - When initializing alpha to zero or close to alpha_max, rho must have a large magnitude

    - Gradient of the sigmoid is near zero -> freezes parameter
    - Increasing T shrinks the input back to zero
- Allows for annealing strategies (e.g., Gumbel-Softmax)

Alpha max parameter to limit the growth of alpha parameter

- Have to be careful with the activation function
- DL likes small numbers
- We want good behavior to be centered around zero (Euclidean), by large numbers could have very large deviation
- Challenging that the simplest behavior can scale in a stable way (because for raw alpha has to be large)
- Maybe using an exponential might be better (might be more stable) because sigmoid is not differentiable at zero

Learnable Alpha – Model Introduction

Canonical Randers Distance (swapping out Euclidean distance)

euclidean_distance(x, y, α)

Else: $d_{euc}((x, y)) = ||x - y||_2$

If alpha ≠ None: $d_{\alpha}((x, y)) = ||x - y||_2 + \alpha(y_D - x_D)$

Corresponds to a Randers metric where the drift vector ω is defined sparsely as $\omega = [0, 0, \dots, \alpha]^T$

1. Triplet Loss
2. Alignment Loss
3. Uniformity Loss
4. Domain-specific Uniformity Loss

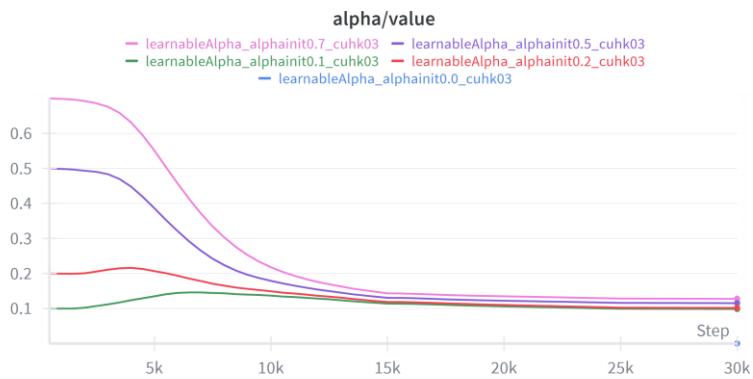
- X and y are two embedding vectors mapped by the network by two different inputs (dimension D)
- Adding alpha adds an asymmetric term based on the difference in the last dimension (D-th dimension)

- Might want to define different distance functions separately
    - We should have at the definition model the association to the distance functions

- Will be migrated into a finsler_distance in the future to accelerate computation on the GPU
    - Also cleaner code structure

Learnable Alpha – Model Introduction

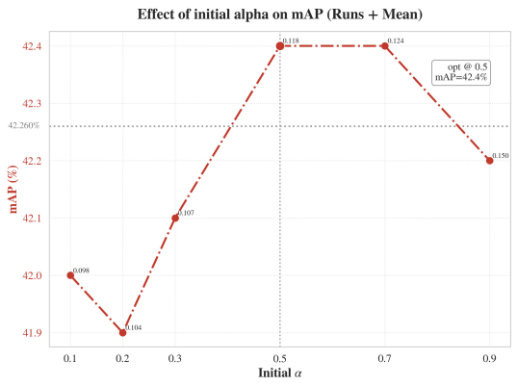
Results

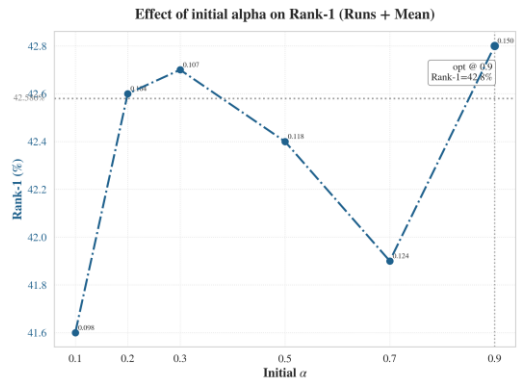


- Also plot the initial rho values (several times to T)
- Rho values is exploding negatively
- Sigmoid curve is becoming flat

Learnable Alpha – Model Introduction

Results





- X and y are two embedding vectors mapped by the network by two different inputs (dimension D)
- Adding alpha adds an asymmetric term based on the difference in the last dimension (D-th dimension)

- This seems like a random seeding because they have the same final value but different mAP and Rank 1

- Will be migrated into a finsler_distance in the future to accelerate computation on the GPU
    - Also cleaner code structure

Shrink latent space to between 0 and 1
Learning a non-constant omega?

Today

1. [**Learnable Alpha**] *Hypothesis*: Raw alpha logit is diverging → final value converges to 0.1
   - Modify alpha's activation function that prevents divergence + known stable behavior at 0
2. [**Learnable Alpha**] Loading a pretrained Euclidean checkpoint + initialize alpha at 0.0
3. [**Learnable Alpha**] Loss ablation test similar to table 5 of the paper
   - *Hypothesis*: Not all losses should use the Finsler distance function or need special care
4. [**Learnable Omega**] Brainstorm and implement architecture to learn drift vector omega
   - Right now the model can only model asymmetry if the "domain shift" happens to align perfectly with the last dimension OR learn to compress all asymmetry to the last dimension

Learnable Alpha – Model Introduction

Alpha Parameter

$0 < \alpha_{max} < 1$ be the maximum scaling factor (default 1.0).

$\rho \in R$ be the **learnable** logit parameter.

$$\alpha = 2 \cdot \boxed{\alpha_{max}} \cdot \left| \boxed{\sigma} \left(\frac{\boxed{\rho}}{\boxed{T}} \right) - 0.5 \right|$$

$\sigma(z) = \dfrac{1}{1 + e^{-z}}$ be the sigmoid function.

$T > 0$ be the temperature scaling factor (default 1.0).

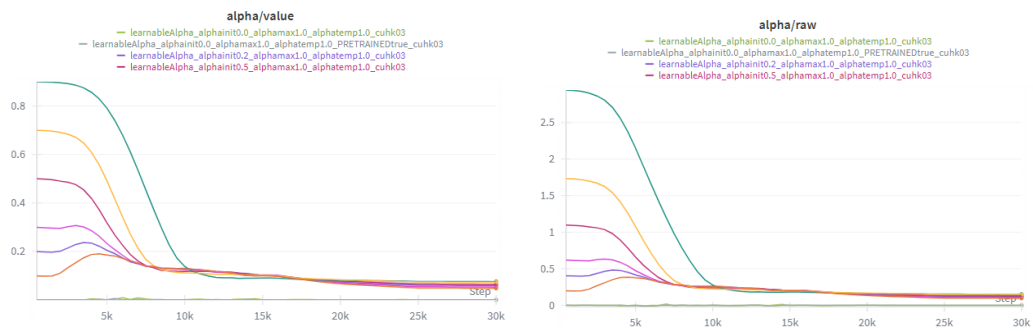
Link to interactive graph

Starting point (original α parameterization)#

- Used a *centered / symmetric* mapping:
- Design goal: **Euclidean baseline at raw=0 ⇒ α=0**
    - Observed issue: **raw=0 becomes a "valley/saddle"**; training can **stick near α≈0** (α doesn't "turn on"), especially when initialized exactly at 0.

Learnable Alpha – Model Introduction

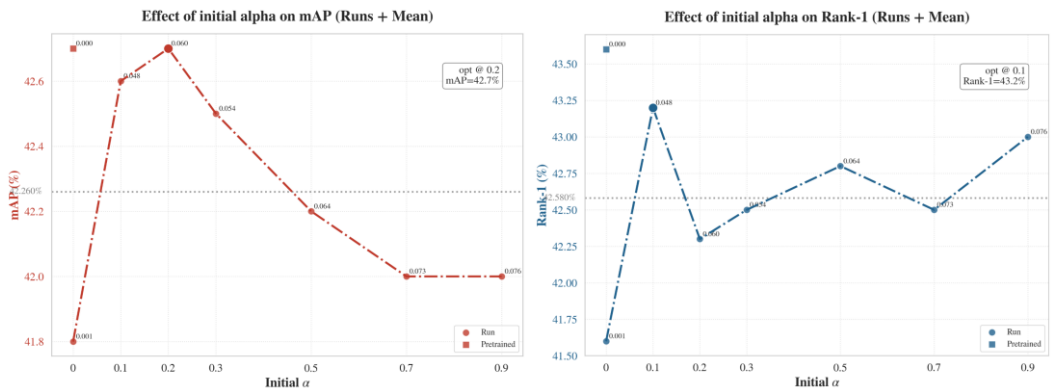
Quantitative Results



Empirical symptom (α-init sweep)

- When initializing **α > 0**, many runs tended to **move toward a small α band** (≈0.1-ish early observations).
- When initializing **α = 0**, α often **failed to leave 0** (or moved very differently), even **without pretrained weights**.
- Interpretation discussed: **gradient starvation / symmetry not broken** → the backbone learns a Euclidean solution that provides little useful signal to grow α.

Learnable Alpha – Model Introduction
Quantitative Results



Alpha values converging lower

- **Old System (Sigmoid):** To reach $\alpha = 0$ (Euclidean baseline), the raw parameter had to be pushed towards $-\infty$. As the value got smaller (e.g., reaching 0.1), the gradients of the sigmoid function naturally vanished. The optimizer essentially "ran out of gas" at 0.1, unable to push the value any lower. 0.1 was likely a **false floor** caused by gradient saturation.
- **New System (Centered):** To reach $\alpha = 0$, the raw parameter simply needs to be 0. The gradient at this point is healthy and non-zero. The model is now free to go lower than 0.1 because the mathematical resistance has been removed.

Pretrained checkpoint behavior

- Loading a **Euclidean pretrained BAU** made it even more likely that α would not increase meaningfully.
- Hypothesis: the embedding space is already optimized for Euclidean distances; turning on asymmetry **initially harms loss**, so α gets no consistent push.

Activation-function brainstorming (goal: keep "stable point" at raw=0 but

avoid dead gradient)

- Considered switching to back **monotonic sigmoid** mapping but clamped the alpha init at 1e-3 (e.g., if we initialize alpha at 0)
- Observed result: In practice, "α-init=0.001" still produced **very negative raw** (≈ −9), pushing sigmoid into **saturation** ⇒ gradients effectively **near-zero** again.
  Net effect: **still "dead" / stuck** (function not helping escape the valley).

Optimizer-side mitigation discussed

- Proposed: **separate optimizer parameter group for α**
  - **Higher LR** for α (e.g., ×100)
  - **No weight decay** on α
- Rationale: α is a single scalar competing with millions of backbone params; needs stronger step size and fewer regularization forces.
- Observation: α converged to zero, performance still subpar

Learnable Alpha – Loss Ablation Study

Introducing the Loss Configurations

Loss Names	Loss Configurations	Aug. Prob.
1_Abs_Baseline	$\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{CE}} + \mathcal{L}_{\text{tri}}$	0
2_Pure_Triplet	$\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{tri}}$	0.5
3_Std_Baseline	$\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{CE}} + \mathcal{L}_{\text{tri}}$	0.5
4_CE_Align	$\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{CE}} + \mathcal{L}_{\text{align}}$	0.5
5_Base_Align	$\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{CE}} + \mathcal{L}_{\text{tri}} + \mathcal{L}_{\text{align}}$	0.5
6_Base_Uniform	$\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{CE}} + \mathcal{L}_{\text{tri}} + \mathcal{L}_{\text{uniform}}$	0.5
7_Base_Domain	$\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{CE}} + \mathcal{L}_{\text{tri}} + \mathcal{L}_{\text{domain}}$	0.5
8_Base_Align_Uni	$\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{CE}} + \mathcal{L}_{\text{tri}} + \mathcal{L}_{\text{align}} + \mathcal{L}_{\text{uniform}}$	0.5
9_Base_Align_Dom	$\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{CE}} + \mathcal{L}_{\text{tri}} + \mathcal{L}_{\text{align}} + \mathcal{L}_{\text{domain}}$	0.5
10_Base_Uni_Dom	$\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{CE}} + \mathcal{L}_{\text{tri}} + \mathcal{L}_{\text{uni}} + \mathcal{L}_{\text{domain}}$	0.5
11_Full_Method	$\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{CE}} + \mathcal{L}_{\text{tri}} + \mathcal{L}_{\text{align}} + \mathcal{L}_{\text{uni}} + \mathcal{L}_{\text{domain}}$	0.5

For the finsler model, we set the following hyperparameters:
ALPHA_INIT=0.1 | ALPHA_MAX=1.0 | ALPHA_TEMP=1.0

To strictly evaluate the impact of the learnable Finsler parameter (alpha) on the backbone's behavior, we avoid the complex regularizers (Align, Uniform, Domain) initially and slowly re-introduce them.

- This isolates whether alpha is aiding the core metric learning (Triplet) or the regularization tasks.

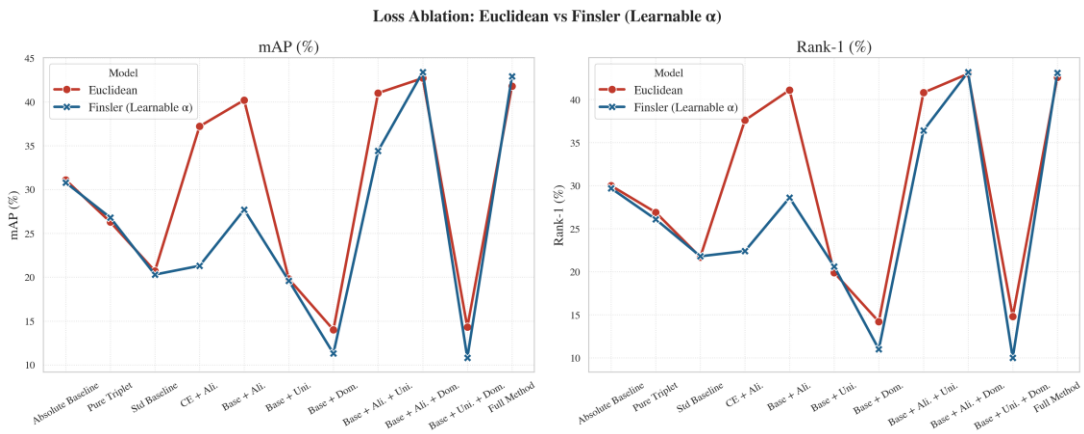
Why alpha_init = 0.1?

- **Convergence Evidence:** The alpha/raw line chart is the most telling. Runs initialized at high values (0.9, 0.7, 0.5) all rapidly collapse and converge towards the **0.1 - 0.2 range**. This suggests the model naturally prefers a small amount of asymmetry but rejects strong asymmetry.
- **Performance & Stability:** The scatter plot ("Effect of Finsler alpha") shows a distinct peak in mean mAP and Rank-1 at 0.1.
- **The "Cold Start" Problem:** The run initialized at 0.0 (grey line) stays very flat near zero. Since alpha acts as a multiplier in the distance addition, if it starts at exactly 0.0, the gradients might be too weak to nurture it away from the Euclidean saddle point. Starting at 0.1 places the parameter in the "active" zone immediately, which should actually **reduce variance** by preventing runs from getting "stuck" in Euclidean space.

1. **Absolute Baseline (No Augs):** ($L_{ce} + L_{tri}$.(The control variable.
2. **Pure Metric:** (L_{tri} only). Critical check: Does alpha help ranking when classification loss (L_{ce}) is removed?
3. **Standard ReID:** ($L_{ce} + L_{tri}$ + Augmentations). The standard baseline for modern ReID.
4. **Base + Align:** Adds geometric alignment. Does alpha help align variations?
5. **Base + Uniform:** Adds uniformity. Does alpha help spread features?
6. **Base + Domain:** Adds domain adaptation. Does alpha help bridge domains?
7. **Base + Align + Uniform:** Testing the core "BAU" hypothesis without Domain loss.
8. **Base + Align + Domain:** Testing alignment consistency across domains.
9. **Base + Uniform + Domain:** Testing distribution shape across domains.
10. **Full Method:** All losses enabled.

Learnable Alpha – Loss Ablation Study

Quantitative Results – Rank 1 and mAP

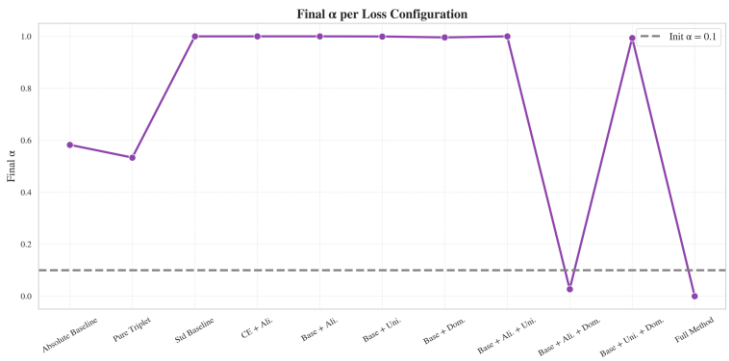


1. **Finsler (learnable α) does not improve overall performance.**
   Across most configurations, Finsler is worse than Euclidean, sometimes substantially (e.g., CE + Align, Base + Align, Base + Align + Uni). The only clear exception is Base + Align + Domain, where Finsler is slightly better.
2. **α behavior is unstable and often saturates.**
   Final α values collapse to 1.0 for many configs (Std Baseline, CE+Align, Base+Align, Base+Uni, Base+Dom, Base+Align+Uni, Base+Uni+Dom). This saturation correlates with performance drops, suggesting the model over-activates asymmetry without benefiting discriminability.
3. **When Finsler performs well, α is near 0.0.**
   The best Finsler results align with α≈0 (e.g., Full Method, and Base+Align+Dom with α≈0.027). This implies the model prefers Euclidean geometry for most loss combinations.
4. **Align loss is harmful under Finsler.**
   Configs that include Align (especially CE+Align and Base+Align) degrade sharply with learnable α, indicating incompatibility between alignment regularization and asymmetric distance (at least in the current formulation).
5. **(Triplet-only does not benefit from Finsler.)**

(Pure Triplet is slightly worse under Finsler, indicating α does not provide a stable metric advantage in isolation.)

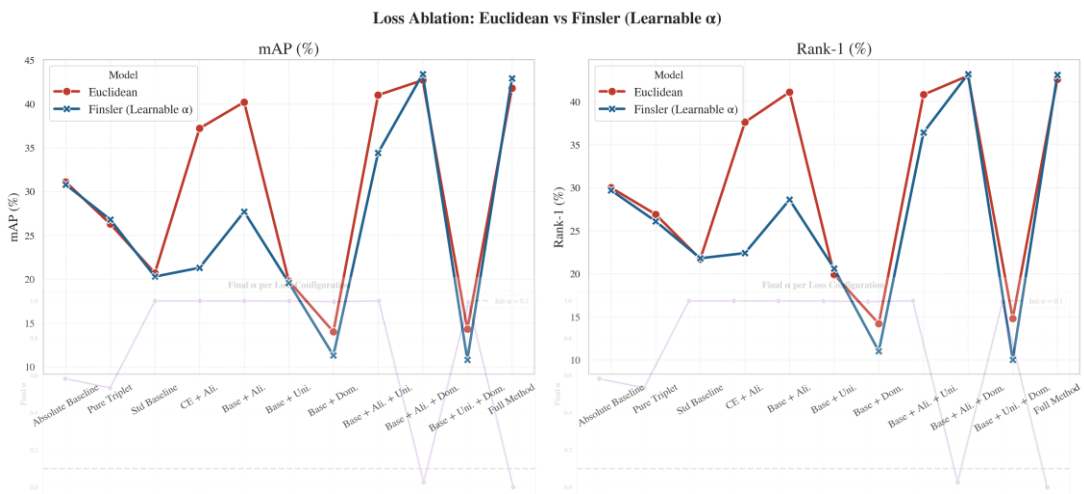
Learnable Alpha – Loss Ablation Study

Quantitative Results – Progression of Alpha



Learnable Alpha – Loss Ablation Study

Quantitative Results – Overlayed



Learnable Alpha – Loss Ablation Study

Conclusions

1. **Finsler (learnable α) does not improve overall performance.**
   Across most configurations, Finsler is worse than Euclidean, sometimes substantially (e.g., CE + Align, Base + Align, Base + Align + Uni). The only clear exception is Base + Align + Domain, where Finsler is slightly better.
2. **α behavior is unstable and often saturates.**
   Final α values collapse to 1.0 for many configs (Std Baseline, CE+Align, Base+Align, Base+Uni, Base+Dom, Base+Align+Uni, Base+Uni+Dom). This saturation correlates with performance drops, suggesting the model over-activates asymmetry without benefiting discriminability.
3. **When Finsler performs well, α is near 0.0.**
   The best Finsler results align with α≈0 (e.g., Full Method, and Base+Align+Dom with α≈0.027). This implies the model prefers Euclidean geometry for most loss combinations.
4. **Align loss is harmful under Finsler.**
   Configs that include Align (especially CE+Align and Base+Align) degrade sharply with learnable α, indicating incompatibility between alignment regularization and asymmetric distance (at least in the current formulation).
5. **(Triplet-only does not benefit from Finsler.)**
   (Pure Triplet is slightly worse under Finsler, indicating α does not provide a stable metric advantage in isolation.)

Overall Interpretation

The current Finsler formulation does not generalize better than Euclidean in this BAU ablation. The only positive signal is **Base + Align + Domain**, where Finsler slightly helps with low α.

Bottom-line takeaway

Two failure modes were observed:

- **α stays near 0** (no symmetry breaking / no gradient signal).
- **α runs to 1.0** (unstable coupling → worse performance).

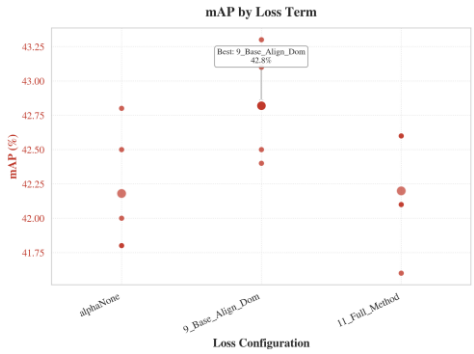
Fixing this likely needs **both** (a) better α parameterization/regularization **and** (b) optimizer treatment (LR/WD) so α learns on an appropriate timescale.

Moving Forward

- **Constrain α** (prevent saturation), or
- **Couple α to specific losses only**, or
- **Rework the drift term so that α>0 is actually beneficial** under alignment/regularization.
- Small learning rate on alpha

Learnable Alpha – Better Without Uniform Loss?

NOTE: It might be the same for the Euclidean case too! Ablation study did not cover this



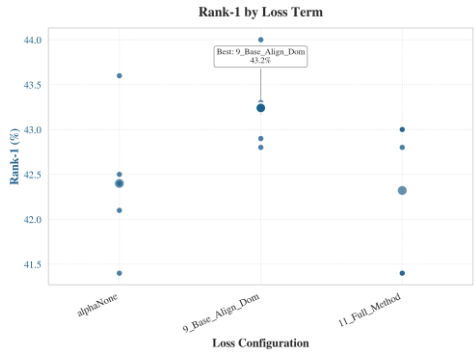


Table 5: Ablation study of loss functions for augmented images.

Method	M+MS+CS → C3		M+CS+C3 → MS		MS+CS+C3 → M		Average	
	mAP	Rank-1	mAP	Rank-1	mAP	Rank-1	mAP	Rank-1
Baseline (w/o augmented images)	39.2	38.9	18.9	44.8	68.3	86.3	42.1	56.7
$\mathcal{L}_{\text{ce}}$	32.1	31.9	18.0	41.7	63.1	83.8	37.7	52.5
$\mathcal{L}_{\text{align}}$	41.8	42.2	23.1	47.4	73.8	87.4	46.2	59.0
$\mathcal{L}_{\text{align}} + \mathcal{L}_{\text{uniform}}$	46.9	47.4	25.3	51.1	78.1	89.5	50.1	62.7
$\mathcal{L}_{\text{align}} + \mathcal{L}_{\text{uniform}} + \mathcal{L}_{\text{domain}}$	**50.6**	**51.8**	**26.8**	**54.3**	**79.5**	**91.1**	**52.3**	**65.7**

Learnable Omega – Model Introduction
Drift Branch

Postponed

Can do it verbally if they want, depending on time

Learnable Omega – Model Introduction

Results

https://wandb.ai/zhuole-lee-technical-university-of-munich/BAU/workspace?nw=znhcqiz5rth

Today

1. [**Learnable Omega**] Brainstorm and implement architecture to learn drift vector omega
   - Implement some fix such that the model is more responsive to alignment loss
2. [**Visualisation**]
   - We could plot pairs of images that have the highest magnitude of omega, and pairs with the smallest magnitude
   - Reduce the IDs in the query so that it is more manageable (perhaps only 10-15)
   - Find a way to keep these IDs always the same across runs (just for the visualization), and not dependent on how the dataset is shuffled given the seed
3. [**Dataset**] Integrate cross-view dataset AG-ReID v2
4. [**Literature**] Re-visit the narrative

Important uni deadline next Thursday, possible
to change meeting to Friday afternoon?
9:45